	Colegio Universitario de Cartago	
	BIG DATA (BD)	

PROYECTOS DE INTELIGENCIA ARTIFICIAL APLICADA

Curso: BD - 151 Inteligencia Artificial Aplicada

Profesor: Osvaldo Gonzalez Chaves

Año 2026

	Colegio Universitario de Cartago	
	BIG DATA (BD)	

Índice

1. Introducción
2. Objetivos Generales
3. Proyecto A: Sistema de Predicción de Riesgo Crediticio
4. Proyecto B: Sistema de Predicción de Abandono de Clientes (Churn)
5. Proyecto C: Sistema de Predicción de Rendimiento Académico
6. Proyecto D: Sistema de Predicción de Calidad de Vino
7. Proyecto E: Sistema de Predicción de Demanda de Bicicletas
8. Estructura de Carpetas del Proyecto
9. Rúbrica de Evaluación

	Colegio Universitario de Cartago	
	BIG DATA (BD)	

1. Introducción

Este documento presenta cinco proyectos integradores de Inteligencia Artificial Aplicada diseñados para grupos de 3 estudiantes. Cada proyecto aborda un problema del mundo real utilizando redes neuronales artificiales (ANN) y requiere el desarrollo de un sistema completo desde la exploración de datos hasta el despliegue con API y frontend.

Los proyectos están diseñados con complejidad equivalente pero en diferentes dominios de aplicación: finanzas, marketing, educación, industria alimentaria y movilidad urbana. Todos utilizan datasets reales y públicos.

2. Objetivos Generales

Al completar estos proyectos, los estudiantes serán capaces de:

- Realizar análisis exploratorio de datos (EDA) con visualizaciones profesionales
- Implementar preprocesamiento de datos para modelos de machine learning
- Diseñar, entrenar y evaluar redes neuronales artificiales (ANN)
- Comparar múltiples modelos y seleccionar el óptimo según métricas
- Desplegar modelos en producción mediante API REST con FastAPI
- Crear interfaces de usuario con Streamlit
- Documentar proyectos de forma profesional

3. Proyecto A: Sistema de Predicción de Riesgo Crediticio

3.1 Descripción del Problema

Desarrollar un sistema inteligente que prediga el riesgo crediticio de clientes bancarios para apoyar decisiones de aprobación de préstamos. El sistema debe clasificar clientes en categorías de riesgo y proporcionar predicciones binarias de aprobación/rechazo.

3.2 Dataset

Dataset: German Credit Data (UCI Machine Learning Repository)

URL: https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data

Registros: 1,000 clientes

Variables: 20 (numéricas y categóricas)

Variables principales: duración del crédito, monto, historial crediticio, propósito, empleo, edad, estado civil, número de créditos existentes, etc.

3.3 Modelos a Implementar

- Modelo 1 - Clasificación Binaria: Predecir aprobación de crédito (Bueno/Malo)
- Modelo 2 - Clasificación Multiclase: Clasificar nivel de riesgo (Bajo/Medio/Alto/Crítico). El dataset solo incluye la etiqueta Bueno/Malo, por lo que el grupo debe definir y justificar el criterio para construir los cuatro niveles

3.4 Entregables Específicos

- Notebook de EDA con análisis de distribuciones y correlaciones
- Notebook de preprocesamiento con variables dummy (get_dummies) y normalización (MinMaxScaler)
- Dos modelos ANN entrenados y guardados (.keras)
- API REST con endpoints /predict/binary y /predict/risk_level
- Frontend Streamlit con formulario de entrada y visualización de resultados
- README.md con documentación completa

4. Proyecto B: Sistema de Predicción de Abandono de Clientes

4.1 Descripción del Problema

Desarrollar un sistema que prediga qué clientes tienen mayor probabilidad de abandonar un servicio de telecomunicaciones. El sistema debe identificar clientes en riesgo y clasificarlos por nivel de urgencia para implementar estrategias de retención.

4.2 Dataset

Dataset: Telco Customer Churn

URL: https://www.kaggle.com/datasets/blastchar/telco-customer-churn

Registros: 7,043 clientes

Variables: 21 (demográficas, servicios contratados, información de cuenta)

Variables principales: antigüedad del cliente, tipo de contrato, método de pago, cargos mensuales, servicios adicionales (internet, streaming, seguridad), etc.

4.3 Modelos a Implementar

- Modelo 1 - Clasificación Binaria: Predecir si el cliente abandonará o no (Churn Yes/No)
- Modelo 2 - Scoring de Riesgo: Estimar la probabilidad de churn (0.0 a 1.0) con una arquitectura distinta a la del Modelo 1 y clasificar por nivel de riesgo según umbrales justificados por el grupo

4.4 Entregables Específicos

- Notebook de EDA con análisis de tasas de churn por segmento
- Notebook con manejo de datos desbalanceados (class_weight)
- Dos modelos ANN: clasificación binaria y scoring de riesgo
- API REST con endpoints /predict/churn y /predict/risk_score
- Dashboard Streamlit con segmentación de clientes por nivel de riesgo
- Análisis de ROI potencial de estrategias de retención

5. Proyecto C: Sistema de Predicción de Rendimiento Académico

5.1 Descripción del Problema

Desarrollar un sistema que prediga el desempeño académico de estudiantes y detecte tempranamente riesgo de deserción. El sistema debe proporcionar alertas tempranas para implementar intervenciones educativas oportunas.

5.2 Dataset

Dataset: Student Performance Dataset (UCI)

URL: https://archive.ics.uci.edu/dataset/320/student+performance

Registros: 395 estudiantes (matemáticas) + 649 (portugués)

Variables: 33 (demográficas, sociales, escolares)

Variables principales: edad, educación de padres, tiempo de estudio, faltas, actividades extracurriculares, apoyo familiar, calificaciones previas, etc.

5.3 Modelos a Implementar

- Modelo 1 - Regresión: Predecir calificación final esperada G3 (0-20). Comparar el desempeño con y sin las notas parciales G1/G2, ya que su uso reduce el valor de la alerta temprana
- Modelo 2 - Clasificación Multiclase: Clasificar nivel de riesgo académico (Sin riesgo/Riesgo bajo/Riesgo medio/Riesgo alto), definido por rangos de G3 justificados por el grupo

5.4 Entregables Específicos

- Notebook de EDA con análisis de factores que impactan el rendimiento
- Feature engineering: creación de variables derivadas (promedio de notas, tasa de ausencias)
- Dos modelos ANN: regresión de calificaciones y clasificación de riesgo
- API REST con endpoints /predict/grade y /predict/risk_level
- Dashboard Streamlit con sistema de alertas tempranas
- Recomendaciones automáticas de intervención según perfil del estudiante

6. Proyecto D: Sistema de Predicción de Calidad de Vino

6.1 Descripción del Problema

Desarrollar un sistema que prediga la calidad de vinos a partir de sus propiedades fisicoquímicas para apoyar el control de calidad en la producción. El sistema debe estimar la puntuación de calidad e identificar los lotes de calidad alta.

6.2 Dataset

Dataset: Wine Quality (UCI Machine Learning Repository)

URL: https://archive.ics.uci.edu/dataset/186/wine+quality

Registros: 6,497 vinos (1,599 tintos + 4,898 blancos)

Variables: 11 fisicoquímicas + calidad (puntuación 0-10)

Variables principales: acidez fija y volátil, ácido cítrico, azúcar residual, cloruros, dióxido de azufre, densidad, pH, sulfatos, alcohol, etc.

6.3 Modelos a Implementar

- Modelo 1 - Regresión: Predecir la puntuación de calidad del vino (0-10)
- Modelo 2 - Clasificación Binaria: Predecir si el vino es de calidad alta (≥ 7) Sí/No

6.4 Entregables Específicos

- Notebook de EDA con análisis de correlaciones entre propiedades fisicoquímicas y calidad
- Feature engineering: variable dummy para el tipo de vino (tinto/blanco) al unir ambos conjuntos
- Manejo del desbalance de clases (pocos vinos de calidad alta) con class_weight
- Dos modelos ANN: regresión de calidad y clasificación binaria de calidad alta
- API REST con endpoints /predict/quality y /predict/high_quality
- Dashboard Streamlit para evaluar lotes y comparar vinos tintos y blancos

7. Proyecto E: Sistema de Predicción de Demanda de Bicicletas

7.1 Descripción del Problema

Desarrollar un sistema que prediga la demanda de alquiler de bicicletas compartidas según las condiciones de calendario y clima. El sistema debe estimar la cantidad de alquileres por hora y clasificar el nivel de demanda para planificar la distribución de la flota.

7.2 Dataset

Dataset: Bike Sharing Dataset (UCI Machine Learning Repository)

URL: https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset

Registros: 17,379 registros por hora (años 2011-2012)

Variables: 17 (calendario, clima y conteos de demanda)

Variables principales: temporada, mes, hora, día festivo, día laboral, situación climática, temperatura, sensación térmica, humedad, velocidad del viento, etc.

7.3 Modelos a Implementar

- Modelo 1 - Regresión: Predecir la cantidad de alquileres por hora (cnt), excluyendo casual y registered porque su suma es la variable objetivo
- Modelo 2 - Clasificación Multiclase: Clasificar nivel de demanda (Baja/Media/Alta/Pico), definido por rangos de cnt justificados por el grupo

7.4 Entregables Específicos

- Notebook de EDA con patrones de demanda por hora, temporada y clima
- Feature engineering: variables dummy para temporada, hora y situación climática
- Dos modelos ANN: regresión de demanda y clasificación por nivel
- API REST con endpoints /predict/demand y /predict/demand_level
- Dashboard Streamlit con proyección de demanda por franja horaria
- Recomendaciones de redistribución de la flota según el nivel de demanda

8. Estructura de Carpetas del Proyecto

Todos los proyectos deben seguir esta estructura estándar:

```
Proyecto_IA/
│
├── README.md                    ← Guía completa de instalación y uso del proyecto
├── requirements.txt             ← Dependencias Python (TensorFlow, FastAPI, Streamlit, etc.)
│
├── data/                        ← Directorio de datos
│   ├── raw/                     ← Datos originales sin procesar
│   │
│   └── processed/               ← Datos limpios y preprocesados listos para entrenar
│       ├── train.csv            ← Conjunto de entrenamiento
│       └── test.csv             ← Conjunto de prueba
│
├── notebooks/                   ← Notebooks para experimentación
│   ├── 01_EDA.ipynb             ← Análisis Exploratorio de Datos (visualizaciones)
│   ├── 02_Preprocesamiento.ipynb ← Limpieza, encoding, normalización
│   ├── 03_ANN_Modelo1.ipynb     ← Entrenamiento del primer modelo ANN
│   ├── 04_ANN_Modelo2.ipynb     ← Entrenamiento del segundo modelo ANN
│   └── 05_Comparacion_Modelos.ipynb ← Evaluación y selección del mejor modelo
│
├── src/                         ← Código fuente modular y reutilizable
│   ├── __init__.py              ← Inicialización del paquete Python
│   ├── config.py                ← Configuraciones globales (rutas, parámetros)
│   ├── data_prep.py             ← Funciones de preprocesamiento de datos
│   └── train/                   ← Scripts de entrenamiento
│       ├── __init__.py
```

```
│   ├── model1.py          ← Entrenamiento del modelo 1
│   ├── model2.py          ← Entrenamiento del modelo 2
│   └── utils.py           ← Utilidades compartidas (métricas, gráficas)
│
├── models/                ← Modelos entrenados y objetos serializados
│   ├── model1.keras       ← Modelo 1 guardado (formato Keras)
│   ├── model2.keras       ← Modelo 2 guardado (formato Keras)
│   ├── scaler.pkl         ← MinMaxScaler entrenado
│   └── columnas.pkl       ← Columnas finales tras get_dummies
│
├── api/                   ← API REST con FastAPI
│   ├── main.py            ← Aplicación FastAPI con endpoints
│   ├── schemas.py         ← Modelos Pydantic para validación
│   └── predict.py         ← Lógica de predicción e inferencia
│
└── app/                   ← Frontend web con Streamlit
    ├── Home.py            ← Página principal del dashboard
    └── pages/             ← Páginas adicionales (multipágina)
        ├── 1_Prediccion.py    ← Página de predicciones individuales
        ├── 2_Analisis.py      ← Página de análisis de lotes
        └── 3_Metricas.py      ← Página de métricas y rendimiento
```

8.1 Descripción de Carpetas

data/: Contiene los datos originales y procesados

notebooks/: Jupyter notebooks para exploración y desarrollo

src/: Código fuente modular y reutilizable

models/: Modelos entrenados y objetos de preprocesamiento

api/: API REST con FastAPI

app/: Frontend con Streamlit

9. Rúbrica de Evaluación

Criterio	Puntos	Descripción
EDA y Preprocesamiento	20%	Análisis exploratorio completo, visualizaciones, limpieza de datos, feature engineering
Modelos ANN	30%	Arquitectura justificada (Dropout, EarlyStopping), entrenamiento con validation_split (test solo para la evaluación final), métricas: classification_report y matriz de confusión, o MAE/RMSE según corresponda
Comparación de Modelos	15%	Al menos dos configuraciones por modelo (p. ej. con y sin Dropout) comparadas con curvas de pérdida y métricas; selección justificada del mejor modelo
API Funcional	15%	Endpoints operativos, validación de datos, manejo de errores, documentación automática
Frontend/Despliegue	10%	Interfaz usable, diseño profesional, navegación clara
Documentación	10%	README completo, comentarios en código, conclusiones y recomendaciones
TOTAL	**100%**	

Notas Finales

- Se recomienda usar Git para control de versiones desde el inicio
- La presentación final debe incluir demostración en vivo del sistema funcionando
- Se valorará la creatividad en las visualizaciones y el diseño de la interfaz
- Cualquier duda técnica puede consultarse en horario de clase o por correo electrónico